

Practical Work - AI and Machine Learning

Software to be configured and Python libraries to be installed (on school computers) :

1) Update `scikit-learn`

- In “Search” under Windows, type “prompt” and launch “Anaconda Prompt”.
- In the window that opens, type : `pip install --upgrade --user scikit-learn` and press Enter.
- Wait for the confirmation message “Successfully installed”.

2) Configure Geany :

- Open Geany and create a Python code file, making sure the file extension is “.py”.
- Open the build configuration window in the “Build” menu → “Set Build Commands”.
- Replace the execution command (first column, second to last line) with : `py "%f"`
- Check that Geany can successfully execute a Python command.

In this practical session, there are three topics to complete :

Part 1 : *Introduction to SkLearn.* The goal of this part is to get familiar with the SkLearn library.

Part 2 : *Material classification.* In this part, the goal is to build a material classification model that predicts the type of glass based on its composition.

Part 3 : *Energy consumption and weather data.* In this final part, the goal is to use a dataset built from the Aix campus to study the relationships between energy consumption and various factors (especially weather conditions) and to try to predict the campus energy consumption from these values.

A deliverable must be provided for Parts 2 and 3 (Part 1 is a discovery exercise without deliverable). Each deliverable must include commented code as well as a short report (Word or similar) explaining your approach and results for each point of the topic.

The resource files contain two types of data : starter files (.py) and the required datasets (.csv) :

part1.py : starter file for Subject 1, uses data.csv

data_1.csv : cloud of 1200 2D points (x, y) with class 0 or 1.

part2.py : starter file for Subject 2, uses glass_data.csv

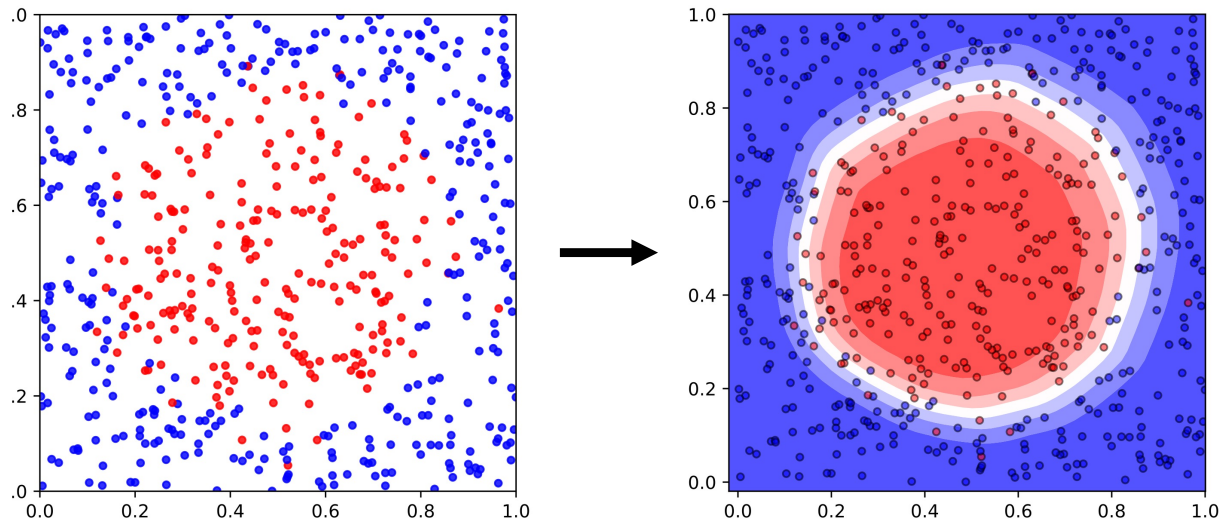
glass_data.csv : characteristics of 214 glass samples and their type.

part3.py : starter file for Subject 3, uses conso_campus_aix.csv

conso_campus_aix.csv : hourly weather and energy consumption data for the Aix campus over one year.

All these files are available on SAVOIR and must be downloaded to complete the exercises.

Subject 1 - Introduction to SkLearn, parameter tuning, test sets, and scoring



Part 1 : Introduction to SkLearn.

The archive contains a file called `part1.py`, which is a simple example that loads the dataset, trains a single-neuron network, and computes the training and testing scores (from the file `data.csv`).

A detailed explanation of the code is available below :

```
import numpy as np
from sklearn.neural_network import MLPClassifier
```

Import the required libraries:
numpy: to load, store, and process our data
sklearn: to build and train neural networks

```
# Loading the data: CSV file with 3 columns and 1200 rows
data = np.genfromtxt('data_1.csv', delimiter=',')
valeurs = data[:, 0:2] # columns 1 et 2
classe = data[:, 2] # column 3
```

`data_1.csv` contains 3 columns: x, y, and class
After loading: `val = [[x1, y1], [x2, y2], [x3, y3], ...]`
After loading: `class = [c1, c2, c3, ...]`

```
# Creating the network
mlp = MLPClassifier(hidden_layer_sizes=(1,), solver='lbfgs', max_iter=10000)
```

hidden_layer_sizes=(1,) for a single layer with one neuron
solver='lbfgs' as the solver
max_iter=10000 for a high maximum number of iterations

```
# Training
mlp.fit(valeurs[0:10], classe[0:10])
```

Run the training on the first 10 data samples using `fit()`

```
# Training score
print("Score train :", mlp.score(valeurs[0:10], classe[0:10]))
# Test set score
print("Score test :", mlp.score(valeurs[10:20], classe[10:20]))
```

Compute the score on the training set (first 10 samples)

Compute the score on the test set (next 10 samples)

The hyperparameters used in this code are not suitable for accurate classification. In this first part, the goal is to make several changes and additions to this program to make it functional and effective.

Before making any changes, check that the code runs correctly and displays two scores (even if they are not high yet).

Several choices must be made to properly tune our model and improve the program :

1. The shape and size of our neural network : the parameter **hidden_layer_sizes** defines the number and size of hidden layers (see documentation [MLPClassifier](#)). A too-small value may lead to underfitting, while too large may cause overfitting. (Values between 1 and 100 are interesting to test here.)
2. The solver : this determines the optimization strategy for weight updates during training. Solvers vary in speed and efficiency depending on the dataset (see doc [MLPClassifier](#)).
3. The activation function : to be chosen depending on the problem type and experimentation (see same doc).
4. The amount of training data : for a simple problem like this, it is not necessary to use the entire dataset. It is interesting to study from what dataset size the score stops improving despite adding more data.
5. A graphical visualization can help understand the impact of parameter choices. The library [Matplotlib](#) is suitable for this purpose. It must first be imported at the top of your program using the line :

```
import matplotlib.pyplot as plt
```

Several types of visualizations can be considered :

- (a) By displaying test results as colored scatter plots using [scatter plot](#).

Example of plotting the first 100 points with colors based on their class :

```
plt.figure() # To be included for each figure
plt.scatter(values[0:100][:, 0], values[0:100][:, 1], c=class[0:100], cmap=plt.cm.bwr, edgecolors='black')
plt.gca().set_aspect('equal', adjustable='box') # Same ratio on X and Y axes
plt.show() # Display (only include once if there are multiple plots)
```

Useful resources : [Matplotlib plot types](#) - [list of colormaps](#) - [scatter plot doc](#)

- (b) By displaying the network's prediction results over the entire data space as a contour map using [contourf](#) :

```
range_xy = np.arange(-0.1, 1.1, 0.05) # Min/max bounds on x and y axes of the plot

image = [] # Background color sampling
for x in range_xy:
    for y in range_xy:
        image.append([x, y])

# Use the trained network on each coordinate of the background image
out = mlp.predict_proba(np.asarray(image))
Z = out[:, 1] # Retrieve the values to build the background image
Z = Z.reshape((len(range_xy), len(range_xy))) # Formatting the background values

plt.contourf(range_xy, range_xy, Z, cmap=plt.cm.bwr, alpha=.6)
plt.scatter(values[:, 0], values[:, 1], c=class, cmap=plt.cm.bwr, edgecolors='black')
plt.show() # display
```

Part 2 : Material classification.



The dataset used in this topic was created by a forensic research center to identify glass fragments (for example, found at a crime scene).

Six types of glass are represented through 214 entries, described by nine features : the refractive index and the percentage composition of eight materials :

8 inputs : Refractive Index K : Potassium **1 output :** 0 : Building window glass (float)
Na : Sodium Ca : Calcium 1 : Building window glass (non-float)
Mg : Magnesium Ba : Barium 2 : Vehicle window glass
Al : Aluminum Fe : Iron 3 : Container glass
Si : Silicon 4 : Tableware glass
5 : Headlamp glass

Example values :

RI	Na	Mg	Al	Si	K	Ca	Ba	Fe	Type
1.52101	13.64	4.49	1.1	71.78	0.06	8.75	0	0	1
1.52369	13.44	0	1.58	72.22	0.32	12.24	0	0	5
...	...	...	...	...	...	...	...	...	...

In the file `part2.py`, the data is loaded and a one-neuron network is trained without a test set. Based on this example, explore the following points :

1. Choose the hyperparameters (see Part 1) and explain your approach.
2. In multi-class classification problems with many input features, it is harder to visualize the results. A confusion matrix is typically used to see which classes are misclassified as others. Implement and display this matrix in your code using the example below :

```
# Required imports
from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# [...] your code here

pred = mlp.predict(valeurs) # Compute the classes for all entries in the list valeurs
cm = confusion_matrix(classe, pred) # Building the confusion matrix

# Display of the confusion matrix
cm_display = ConfusionMatrixDisplay(cm)
cm_display.plot(cmap=plt.cm.Blues)
plt.show()
```

3. In this example, it would be difficult to detect overfitting. To avoid this, add a test set. SkLearn provides a function ([train_test_split](#)) that performs a random train/test split. Example :

```
valeurs_train, valeurs_test, classe_train, classe_test = train_test_split(valeurs, classe, test_size=0.33)
```

What do you observe about the scores after splitting the dataset? Adjust your hyperparameters to achieve a good test score.

4. Perform several trainings (in a Python loop) with new random splits each time to compute average scores over many runs. Which glass class is the hardest to recognize on average?

Part 3 : Energy consumption and weather data.

The Arts et Métiers Aix-en-Provence campus is equipped with energy monitoring systems that record electricity consumption across the campus on an hourly basis. This consumption depends on many factors : building occupancy (day/night/weekend/holidays), ongoing activities, weather conditions, and time of year. Several meteorological variables for Aix have been collected (from Météo-France) to build a combined hourly dataset containing temperature, wind speed, wind direction, and humidity.

All these data have been merged into a single one-year dataset, hour by hour : from October 13, 2021, to October 12, 2022 — that is, 8725 entries (17 missing values due to probable sensor issues).

The dataset structure is as follows :

Day	Month	Year	Hour	Wind speed	Wind direction	Temperature	Humidity	Consumption
13.0	10.0	2021.0	0.0	0.0	0.0	8.3	86.0	50.0
13.0	10.0	2021.0	1.0	0.5	360.0	7.8	87.5	50.0
13.0	10.0	2021.0	2.0	0.0	0.0	7.4	89.0	51.0
...	...	...	...	...	...	...	...	...

The starter file `part3.py` opens this dataset and provides a simple example of using a regression network ([MLPRegressor](#)).

1. To better understand the dataset, we can plot the variables. For example, as line plots :

```
plt.plot(conso)
plt.show() # Display the plot
```

To better visualize temporal evolution, we can use months on the x-axis :

```
month = np.asarray(["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"])
# Retrieve the indexes of the beginning of each month
month_start_index = np.where((values[:, 0] == 1) & (values[:, 3] == 0))[0]
ax = plt.gca()
ax.set_xticks(month_start_index) # One tick for each beginning of the month
# use the string values from the month array at the indices stored in month_start_index
ax.set_xticklabels(month[values[:, 1][month_start_index].astype(int)-1], rotation=45)

ax.plot(conso) # Display the consumption
plt.show() # Display the plot
```

All variables can be displayed by replacing `ax.plot(conso)` with `ax.plot(values[:, i])`, where `i` is the index of the variable to plot.

2. Based on `part3.py`, explore different approaches. The simplest one is to predict the current consumption from the current weather variables. Build a model to make this prediction and choose appropriate hyperparameters.

To reduce convergence instabilities, we can normalize input variables to the range 0–1 :

```
for i in range(8):
    values[:, i] = values[:, i] / np.max(values[:, i])
```

3. Input variables may have different levels of influence. We can study this by removing some variables to see how performance changes.

There are several ways to remove a feature : you can exclude it from the input array, or make it unusable by randomly permuting its values. Here is an example of random permutation on one feature column :

```
# Make a copy of all input entries  
values_2 = np.copy(values)  
# Perform a random permutation on the column with index 0  
values_2[:, 0] = np.random.permutation(values_2[:, 0])
```

Which input variables have the strongest impact on prediction ?

4. Some inputs may be redundant (providing similar information to the network). Try removing multiple features simultaneously. Which combinations are most impactful ? What is the smallest set of inputs that still allows a decent prediction ? Why ?